Progress Report: Soccer Query System

Project Summary

Our goal is to build a soccer analytics query system that is more reliable and interpretable than a weak single-pass baseline. Instead of mapping a natural-language query directly to SQL in one step, the system uses a structured pipeline with parsing, entity resolution, SQL execution, and result validation. This design is particularly useful for harder cases such as multi-season queries, under-specified inputs, and teams that appear under multiple internal IDs.

At this stage, we have a working end-to-end MVP. The current system supports four query types: `match_count`, `home_wins`, `away_wins`, and `goals_scored`. We have also implemented a weak baseline and an automated benchmark runner for side-by-side comparison. The current results already suggest that the structured pipeline is more robust than the baseline, with the largest gains coming from grouped team resolution, multi-season support, and validation-aware handling of ambiguous inputs.

System Design and Current Progress

The current pipeline is:

$$Query \rightarrow Parser \rightarrow Resolver \rightarrow Controller \rightarrow SQLExecutor \rightarrow Validator$$

The parser converts a raw user query into a structured specification containing the main fields needed for execution, including competition, season, team, and query type. At the current stage, the parser is rule-based and covers the four supported query types. Although simple, this separation is useful because it turns an unstructured question into an explicit intermediate representation, making the downstream pipeline easier to inspect, debug, and extend.

The resolver maps parsed entities to database-compatible values. For competitions, it resolves league and season combinations into competition IDs, including cases where a query refers to multiple seasons. For teams, it resolves team names into database team IDs and supports canonical grouping when one real-world team appears under multiple internal IDs. This is an important design choice, because without grouping, the system can miss part of the team's data. In practice, this matters for cases such as Real Sociedad and Atletico Madrid. The resolver also outputs confidence scores, which provide an additional signal about the reliability of entity grounding.

The controller connects all modules into one execution flow. It receives the structured query, calls the resolver, selects the appropriate SQL template based on query type, executes the query through a read-only SQL executor, and then passes the result to the validator. This modular design keeps the role of each component clear and makes failures easier to localize during development.

The validator provides a final quality check on top of raw database execution. Instead of always returning only a numeric result, the system can produce structured outcomes such as `OK`, `CLARIFY`, and `REPAIR`. This makes the pipeline more informative on difficult cases, especially when the user query is ambiguous or only partially specified. For comparison, we also implemented a weak baseline. It uses the same parser, but performs only single-pass resolution and execution, without

grouped multi-ID support, without multi-season support, and without validation. This baseline serves as a simple reference point for measuring the value of the full pipeline.

Evaluation

We evaluated the current system on 16 benchmark cases covering single-season queries, multi-season queries, merged-team cases, under-specified queries, and goal aggregation queries.

System	Success Rate	Expected Status Match	Correctness (Gold Cases)
Baseline	50.00%	75.00%	50.00%
Full System	62.50%	87.50%	100.00%

Table 1: Benchmark comparison between the weak baseline and the full pipeline.

These results show that the full system is already more robust than the baseline. The largest gains come from two places. First, grouped team resolution improves correctness for teams with multiple internal IDs. In the baseline, these cases often return incomplete or incorrect counts because the system grounds the team to only one internal ID. Second, the full system supports multi-season competition queries, while the baseline fails on such inputs.

The validator also adds practical value. For under-specified queries, the system does not simply fail silently or return a misleading result. Instead, it can surface a `CLARIFY` or `REPAIR` decision, which makes the pipeline easier to inspect and more realistic for interactive use.

Case	Query	Baseline	Full System	Main Reason
c2	Real Sociedad home wins	0	8	grouped team IDs
c3	Atletico Madrid away wins	0	8	grouped team IDs
c7	Multi-season query	fail	success	multi-season support
c5	Missing season	fail	`CLARIFY`	validator detects ambiguity

Table 2: Representative examples showing the advantage of the full system.

We also extended the system to support `goals_scored`, which shows that the pipeline is not limited to simple counting queries. This extension is now integrated into the parser, controller, benchmark, and validation pipeline, and it demonstrates that the current architecture can absorb more complex aggregation logic without changing the overall design.

Current Limitations

The current parser is still rule-based, so its language coverage remains limited. The benchmark is also still relatively small, and only a subset of cases currently has gold answers. In addition, the

validator can classify problematic outputs, but it does not yet trigger an automatic repair or retry loop.

Next Steps

The next stage of the project will focus on three directions. First, we plan to replace the current rule-based parser with an LLM-based parser, which better matches the long-term agent-style goal of the system. Second, we will expand the benchmark with more natural paraphrases, more gold-labeled cases, and more edge cases involving ambiguity and multi-season reasoning. Third, we will strengthen the interaction between the controller and validator so that the system can move beyond detection toward automatic repair and relaxation.

Repository

Code repository: `https://github.com/Chris310103/soccer_query_agent`

The repository includes the parser, resolver, controller, validator, weak baseline, and benchmark scripts, together with exported evaluation outputs.